



**Abertay
University**

Malware Analysis

Identification and analysis of a chosen sample of malware

Samuel Rutherford

CMP320: Advanced Ethical hacking

BSc Ethical Hacking Year 3

2022/23

Note that Information contained in this document is for educational purposes.

Abstract

The refinement of malware has led to more critical action being required to take place. Malware achieves this notoriety due to its ability to wreak havoc to establishments of any size and often comes in different forms such as ransomware and worms. It accomplishes this goal by utilising exploits in a victim's machine to lead to unintended privileges being granted, often resulting in the malware taking full control of the victim's machine. This paper aims to perform a comprehensive analysis of a chosen malware sample within a test environment.

The sample provided first underwent static analysis which refers to observing the files of the malware for any signs of malicious intent while it is not running. Various tools were used during the static analysis process including VirusTotal, Dependency Walker and x32dbg. From this, the sample was identified to be the infamous malware known as WannaCry and other information about the inner workings of the executable were identified. Using this information, dynamic analysis was performed with the necessary safety precautions. Process Monitor and Wireshark were used to observe how the sample reacts and behaves in an environment where it can fulfil its objective. The results were recorded and written up in a white paper format.

The testing of the sample conclusively led to the idea that it was a part of WannaCry, specifically the encryption/ransom part, lacking the worm aspect of it. WannaCry was confirmed to have the ability to take control and manipulate a user's machine with its goal of encrypting a user's important files, making the victim pay a ransom to recover the files. The results should be used for future malware analysis and identifying mitigation against WannaCry and other ransomware. Utilising anti-virus software is critical to protecting organisations and home networks against malware and should always be deployed. Due to the time constraints not every aspect of the sample could be analysed and the parts which already were could have been more vigorously tested such as disassembly and function analysis. For example, analysing how the malware interacts with the network could have been improved upon. Overall, the analysis performed was conclusive in identifying what malware was present and its behavioural characteristics.

Contents

1	Introduction	1
1.1	Background	1
1.2	Aims.....	2
1.3	Methodology.....	2
2	Procedure and results	4
2.1	Overview of Procedure	4
2.2	Static analysis	4
2.2.1	Virus total.....	4
2.2.2	DLL analysis	5
2.2.3	Packing software detection	5
2.2.4	Strings.....	6
2.2.5	Extracting the malware	8
2.2.6	X86 assembly analysis	11
2.3	Dynamic analysis.....	14
2.3.1	Running WannaCry	14
2.3.2	Function analysis.....	17
2.3.3	Network analysis.....	18
3	Discussion.....	20
3.1	Overall Discussion	20
3.1.1	Static analysis	20
3.1.2	Dynamic analysis.....	20
3.2	Countermeasures.....	21
3.3	Future Work	22
4	Conclusion	23
5	Bibliography	24
	Appendices part 1	26
	Appendix A.....	26

1 INTRODUCTION

1.1 BACKGROUND

The threat of malware to businesses, governing bodies and the general public is ever-growing. Within the first half of 2022 alone, there was 2.8 billion malware attacks [1] being used to propagate malicious programs such as viruses, worms, and ransomware. The damage that can be caused by malware is devastating to smaller businesses [2] and home networks due to potential ransomware encrypting their files, effectively locking them out unless they pay a certain fee to an individual. Comparatively, malware is also extremely effective in locking down a large-scale organization such as a business or local governments due to many utilizing worms which intend to spread to itself onto many as many devices as it can within the network using various exploits. Malware can achieve all of this by utilizing exploits found within the victim's machine to perform the spreading of itself and gain control of the user's machine.

The analysis of malware allows for cyber security specialists to develop a thorough understanding of what they are against when developing countermeasures. The war against malware developers and cybersecurity/anti-virus vendors is a constantly evolving arms race where knowing your enemy is crucial to success. Often malware developers are one step ahead due to the nature of malware being an offensive tool, so being able to identify the functionality, nature and type of malware found within a sample can greatly help prevent or even stop further spread of the malware.

Being able to understand and identify malware from a sample is incredibly important to maintaining security globally, especially considering 560,000 new pieces of malware are found every day [3], continually adding to the growing number:

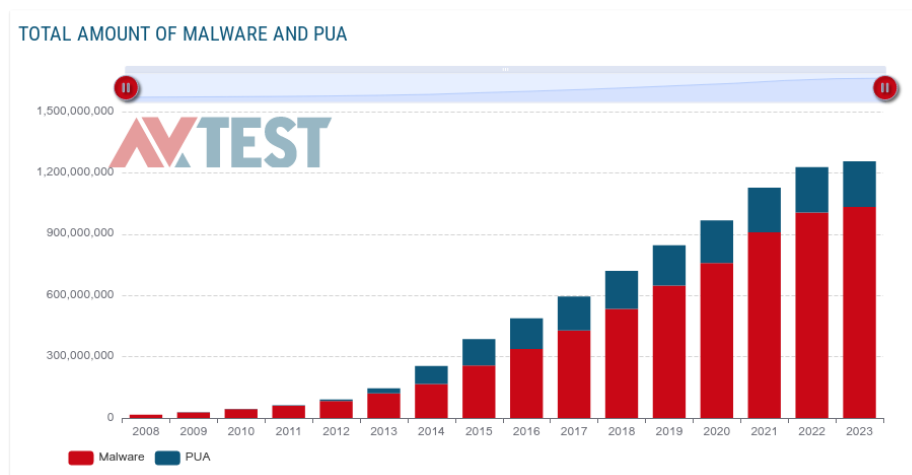


Fig 1–1 Graph displaying the total amount of malware and potentially unwanted applications recorded by AV-test [4].

From the analysis, the ability to understand what a malware is trying to accomplish, similarities or repeating code from other well-known malware and its methods of attempting to propagate itself are all essential to preventing the further spread and damage the malware can achieve and can help prevent zero-day attacks [5].

The following report will go into detail on the static and dynamic analysis performed on a sample provided. Static analysis refers to observing the malware while it has not been executed and identifying any noteworthy characteristics about it. Comparatively, dynamic analysis involves running the malware in a safe environment and noting how it reacts to being within a machine and achieving its goal as a malware, whether it's a worm, ransomware, trojan etc. Finally, an in-depth breakdown of the results, difficulties and countermeasures which can be taken against the malware.

The analysis was performed on a virtual machine to prevent any own files from being damaged or lost and the network type to be "Host only" to prevent the malware from potentially spreading across the network. The samples were provided by Abertay University.

1.2 AIMS

This paper aims to perform a comprehensive analysis of a chosen malware sample within a test environment. The primary aim consists of several smaller aims:

- Identify the type of malware within the sample.
- Perform static and dynamic analysis to evaluate the samples characteristics, components, and functionality.
- Suggest effective countermeasures against the identified malware.

1.3 METHODOLOGY

The analysis was performed using a variety of tools and techniques to best gather results and identify the sample as a malware. All these tools for analysing can be divided into two separate categories which are static and dynamic analysis [6]. Static analysis involves observing the malware while it remains dormant and not currently attempting to cause damage to a machine. Performing static analysis is crucial to an investigation due to many malwares leaving technical hints and provide a great overview of the malware's infrastructure. The tools being used for static analysis include:

- VirusTotal website for providing an estimated guess as to what the sample was.
- Dependency Walker for viewing the sample's DLLs.
- PeID for identifying the packing software used.
- Strings.exe for observing strings within the sample.
- X32dbg for further string and x86 disassembly analysis.

Dynamic analysis provided an opportunity to view how exactly the malware behaves and observe it's characteristics. Due to the malware's goal of causing damage to a machine [7], performing dynamic analysis in a safe test environment is crucial, which is why the following tools were used in a virtual machine:

- Process Monitor for identifying the functions being created and exited.
- Wireshark to analyse how the malware interacts with the network.

The results are recorded and then suitably written up in a white paper report format.

2 PROCEDURE AND RESULTS

2.1 OVERVIEW OF PROCEDURE

To begin the procedure, a total of 8 samples were provided by the university all within zip folders. Sample 1 was chosen to be further analysed since, once the type of malware had been identified, the malware was recognisable and had many resources surrounding it. The tools used for the malware analysis were provided on the virtual machine included with the samples and sources online for verifying the characteristics and functions of the malware.

2.2 STATIC ANALYSIS

2.2.1 Virus total

An effective way to immediately verify what the malware is in a sample was to upload it to VirusTotal.com. VirusTotal is a website which compares and analyses files and checks them for potential malware which may be present [8]. Due to the nature of the virtual machine not being connected to the network and moving malware is exceptionally dangerous, it is possible to upload a hash of the sample to also perform analysis. Numerous hashing algorithms were performed on sample 1 giving the results:

Hashing algorithm	Hash
MD5	84C82835A5D21BBCF75A61706D8AB549
SHA1	5ff465afaabcbf0150d1a3ab2c2e74f3a4426467
SHA256	ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa

Table 2-1 Hashes for sample 1.

The MD5 hash was then uploaded to VirusTotal:

65 / 68

65 security vendors and 5 sandboxes flagged this file as malicious

ed01ebfbc9eb5bbea545af4d01bf5f1071661840480439c6e5babe8e080e41aa

3.35 MB Size

2023-03-30 10:42:25 UTC 3 hours ago

EXE

peexe malware macro-create-ole self-delete overlay runtime-modules detect-debug-environment checks-network-adapters spreader long-sleeps direct-cpu-clock-access via-tor calls-vmm

checks-user-input checks-disk-space executes-dropped-file

Community Score

DETECTION DETAILS RELATIONS BEHAVIOR COMMUNITY 30 +

Join the VT Community and enjoy additional community insights and crowdsourced detections, plus an API key to automate checks.

Popular threat label 65 ransomware.wannacry/wannacryptor Threat categories ransomware trojan Family labels wannacry wannacryptor wannacrypt

Security vendors' analysis Do you want to automate checks?

Fig 2–1 MD5 hash of sample 1 uploaded to VirusTotal, believed to be WannaCry.

According to VirusTotal, the sample is believed to be the well-known ransomware and worm, known as wannacryptor or WannaCry. VirusTotal was able to identify WannaCry using 70 different security vendors including F-Secure, Google and Kaspersky.

WannaCry is an infamous ransomware software which is used as a method for cybercriminals to take money from victims who accidentally install the software [9]. This is done by WannaCry encrypting the victims' files, essentially locking them out. The only way to recover the files is to send money (in the cryptocurrency, 'bitcoin') to the criminals who then decrypt your files. WannaCry spreads itself through the network using the eternal blue exploit, an exploit which utilises SMB packets to propagate itself.

2.2.2 DLL analysis

DLL (Dynamic Link Library) are libraries which can be shared by multiple different programs at the same time. Within a DLL, it can be expected to find functions, classes, and variables which other executables can use [10]. This relates to malware analysis as the DLLs which can be found within a sample can hint at the capabilities and goals of the malware.

Using dependency walker, the DLLs found within the WannaCry sample can be found:

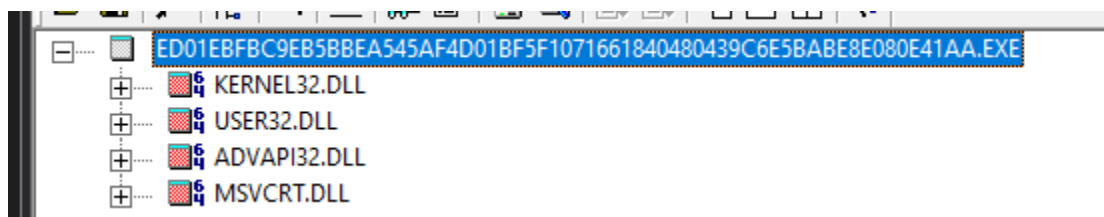


Fig 2–2 DLLs found within Sample1.

4 large DLLs can be found within WannaCry which all contribute to some aspect of the malware. The '32' within the names, indicate that they are 32 bit.

Kernel32.DLL is a common DLL which is used for the access and manipulation of memory and files [11]. It is a very common DLL and a Windows Kernel Module which indicates that it works very closely to the operating system. The memory and file manipulation capabilities indicate that the malware can take control of the users' machine and cause damage. These function capabilities would allow for file encryption to take place, which is an expected behaviour from WannaCry.

USER32.DLL manages UI aspects of a machine including buttons, scroll bars and other components which the user interacts with [12]. The malware can use USER32.DLL to potentially lock out the user by severely restricting their use of UI components, making any file recovery, even trickier.

ADVAPI32.DLL gives access to core Windows components including service manager and the registry. A malware can use services to remain persistent and use the registry to cause more damage on the host's machine.

MSVCRT.DLL contains many C library functions [13]. This is the first indication that the malware is packed, most likely with a software involving a C library.

2.2.3 Packing software detection

Malware often uses packing as a method to avoid detection. PEID can be used to identify the packing software used:

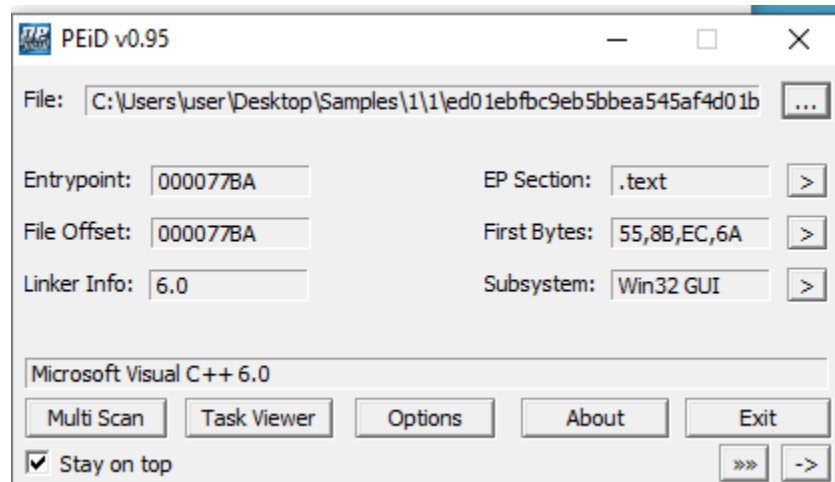


Fig 2–3 PEiD used on sample 1, found to be using C++ 6.0 as a packing software.

The malware is found to be using Microsoft Visual C++ 6.0, which can be expected due to finding MSCVRT.DLL prior. The packing acts as an additional layer of code which further obscures the malware from any researchers trying to further break it down. It is impossible to get passed the packing software unless the analyst knows what packing software is being used.

2.2.4 Strings

Observing the strings found within malware can greatly hint at what the malware is trying to accomplish. The strings found can often be file names, functions or executables and is an effective method of finding a starting point for an investigation. Running Strings, a tool which displays this information on WannaCry displays the following output:

```
taskdl.exe
Jy5
taskse.exe
ln$D
u.wnry
PAD
VS_VERSION_INFO
StringFileInfo
040904B0
CompanyName
Microsoft Corporation
FileDescription
DiskPart
FileVersion
6.1.7601.17514 (win7sp1_rtm.101119-1850)
InternalName
diskpart.exe
LegalCopyright
Microsoft Corporation. All rights reserved.
OriginalFilename
diskpart.exe
ProductName
Microsoft
Windows
Operating System
ProductVersion
6.1.7601.17514
VarFileInfo
Translation
```

Fig 2–4 First output from sample1 using strings.

Numerous executable files can be found. Taskdl.exe is used for the removal of temporary files and performs a general clean-up of the victims' machine once the encryption has taken place and Taskse.exe launches the decryption tool [14]. Additionally, the first instance of a file with the using the '.wnry' extension is found, being 'u.wnry'. The '.wnry' is an extension which is only typically found within WannaCry, it is usually used some parts of the malwares configuration and deployment of itself. These files become more common and will be elaborated in depth later.

```
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <trustInfo xmlns="urn:schemas-microsoft-com:asm.v2">
    <security>
      <requestedPrivileges>
        <requestedExecutionLevel level="asInvoker" />
      </requestedPrivileges>
    </security>
  </trustInfo>
  <dependency>
    <dependentAssembly>
      <assemblyIdentity
        type="win32"
        name="Microsoft.Windows.Common-Controls"
        version="6.0.0.0"
        processorArchitecture="*"
        publicKeyToken="6595b64144ccf1df"
        language="*"
      />
    </dependentAssembly>
  </dependency>
  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
    <application>
      <!-- Windows 10 -->
      <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}" />
      <!-- Windows 8.1 -->
      <supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}" />
      <!-- Windows Vista -->
      <supportedOS Id="{e2011457-1546-43c5-a5fe-008deee3d3f0}" />
      <!-- Windows 7 -->
      <supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}" />
      <!-- Windows 8 -->
      <supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}" />
    </application>
  </compatibility>
</assembly>
PPADDINGXXPPADDINGPPADDINGXXPPADDINGPPADDINGXXPPADDINGPPADDINGXXPPADDINGPPADDINGPPADDINGXXPPADDING
```

Fig 2–5 Manifest found towards the end of the Strings output.

At the end of the Strings output, the manifest is found. This also matches the output from pestudio which can be found in the appendix. The manifest file is an XML file which contains metadata for the program to use. Manifests are necessary to be within malware due to providing essential functionality and additionally how they can cooperate with different versions of DLLs, which WannaCry make use of.

For easier and more interactive use, the tool being used for searching for strings was switched to x32dbg, a debugging tool. Using breakpoints, the entry point can be located, alongside the primary module.

Type	Address	Module/Label/Exception	State	Disassembly	Hits	Summary
Software	004077BA	<e01ebfbc9eb5bbea545af4d01bf5f1071	One-time	push ebp	0	entry breakpoint

Fig 2–6 Finding the breakpoint in the sample.

With the main module being viewed, the strings can now be accessed easily and searching for specific strings is much easier.

```
String
L"%s\\%s"
L"%s\\ProgramData"
L"%s\\Intel"
"cmd.exe /c \"%s\""
"%s%d"
"%s%s%s"
"%s%s"
"%s"
"iled 0x%08lx\n"
L"1 has been restricted by your Administrator by policy rule %2.\r\n"
L"d in the dynamic link library %hs.\r\n"
```

Fig 2–7 String searching for ‘%’ in sample1.

The %s refer to placeholder strings within the executable.

```
"cmd.exe /c \"%s\""
"XIA"
"C.wnry"
"13AM4VW2dhxYgXeQepoHkHSQuy6NgaEb94"
"12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw"
"115p7UMMngoJ1pMvKpHiJcRdfJNXj6LrLn"
"Global\\MSWinZonesCacheCounterMutexA"
"%s%d"
"taskche.exe"
```

Fig 2–8 Located strings of text which resemble bitcoin wallets.

Within the string references, 3 strings which appear to be bitcoin wallets can be found. This is to be expected due to how the random is paid, is by using bitcoin.

```
"tasksche.exe"
"tasksche.exe"
"WNcry@2017"
"attrib +h ."
```

Fig 2–9 String resembling a password is found.

A string which could be resembling a password is found. The exact password is “WNcry@2017” using a mixture of special characters, numbers, and letters to further confuse any analysts. The most likely use for that password would be to unzip a zipped folder.

2.2.5 Extracting the malware

Using the password acquired before, it is possible to unzip the sample. Doing so results in the following files being added to the same destination as the sample:

.....				
ples > 1 > 1 > wannacry >				
Name	Date modified	Type	Size	
msg	4/20/2023 3:03 PM	File folder		
b.wnry	5/11/2017 12:13 PM	WNRY File	1,407	KB
c.wnry	5/11/2017 12:11 PM	WNRY File	1	KB
r.wnry	5/11/2017 7:59 AM	WNRY File	1	KB
s.wnry	5/9/2017 8:58 AM	WNRY File	2,968	KB
t.wnry	5/11/2017 6:22 PM	WNRY File	65	KB
taskdl.exe	5/11/2017 6:22 PM	Application	20	KB
taskse.exe	5/11/2017 6:22 PM	Application	20	KB
u.wnry	5/11/2017 6:22 PM	WNRY File	240	KB

Fig 2–10 File contents of the unzipped sample.

The folder contains many files such as the executables (taskdl.exe and taskse.exe) found in the strings before. Additionally, there is a whole other folder known as 'msg' which contains the ransom note in a variety of languages from Portuguese to Mandarin:

```
h\af53\insrsid14313477\charrsid5733561 \hich\af31502\dbch\af53\loch\af31502 ure. We guarantee that you can recover all y\hich\af
31502 You can decrypt some of your files for free. Try now by clicking <Decrypt>.
31502 But if you want to decrypt all your files, you need to pay.
31502 You only have 3 days to submit the payment. After that the\hich\af31502\dbch\af53\loch\af31502 price will be doubled.
31502 Also, if you don't pay in 7 days, you won't be able to recover your files forever.
31502 We will have free events for users who are so poor that they couldn't pay in 6 months.
```

Fig 2–11 Ransom note from 'm_english.wnry' opened using 010 Editor.

More information on the nature of the ransom and recovery/decryption of the files is revealed. The victim has 3 days to pay \$300 in bitcoin before the ransom is doubled and then on the 7th day the files are lost forever. All of the message files end in the '.wnry' extension, further reinforcing that the extension does not just appear on the encrypted files.

The rest of the unzipped contents appear to be other files with the '.wnry' extension. The first file was 'b.wnry' which, when changing the extension to '.bmp', allows for an image to become visible:

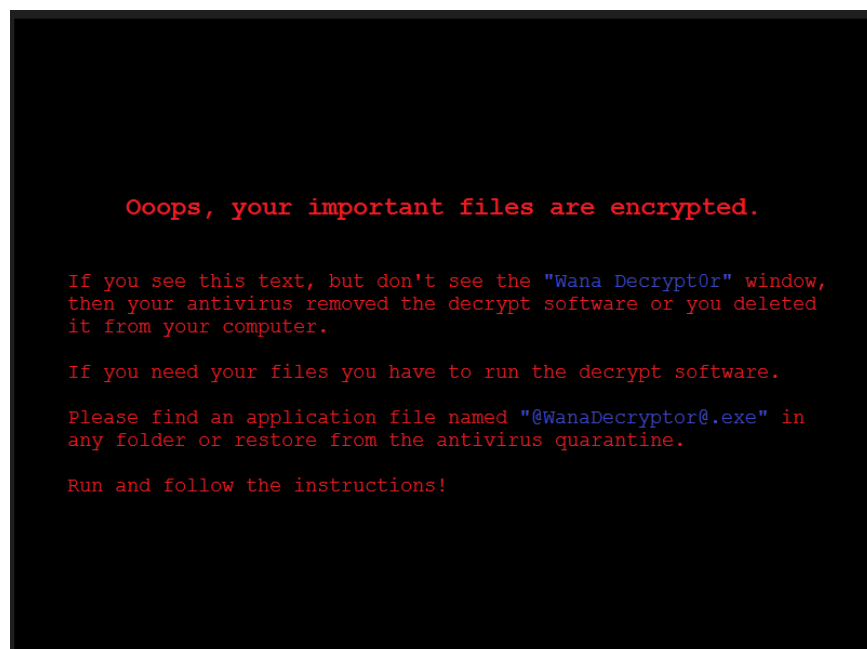


Fig 2–12 Image of 'b.wnry'.

The image shown is what the desktop background is changed to when a victim runs WannaCry. On the image, there are mentions of "Wana Decrypt0r" which contains more information about the ransom payment.

The next file is 'c.wnry' which can be viewed using 010 editor:

```

0 .....
9 ....gx7ekbenv2ri
3 ucmf.onion;57g7s
F pgrzlojinas.onio
2 n;xxlvbrloxyriy2
9 c5.onion;76jdd2i
B r2embyv47.onion;
7 cwwnhwhlz52maq7
0 .onion;.....
0 .....

```

Fig 2–13 Contents of 'c.wnry'.

The contents of 'c.wnry' may appear to be random numbers and letters however they are onion addresses. Onion addresses make use of Tor, which an acronym for 'The onion router', which makes use of a layered encryption to fully anonymise the traffic being sent by an individual [15]. Tor achieves this by using multiple routers to 'bounce' around which every time there's a 'bounce', the data is further encrypted [16]. Additionally, Tor does not rely on typical domains to navigate the web, allowing for access to blacklisted sites. The onion addresses are constructed using a random string of 16 to 56 characters followed by '.onion', acting as the domain. These addresses when used alongside Tor are most likely used for C2 (Command and control) capabilities of the malware.

Viewing 'r.wnry' in 010 editor reveals the following text:

```

Q: What's wrong with my files?
A: Ooops, your important files are encrypted. It means you will not be able to access them anymore until they are decrypted.
   If you follow our instructions, we guarantee that you can decrypt all your files quickly and safely!
   Let's start decrypting!
Q: What do I do?
A: First, you need to pay service fees for the decryption.
   Please send %s to this bitcoin address: %s
   Next, please find an application file named "%s". It is the decrypt software.
   Run and follow the instructions! (You may need to disable your antivirus for a while.)
Q: How can I trust?
A: Don't worry about decryption.
   We will decrypt your files surely because nobody will trust us if we cheat users.
* If you need our assistance, send a message by clicking <Contact Us> on the decryptor window.

```

Fig 2–14 Contents of 'r.wnry'.

Within 'r.wnry', there appears to be a FAQ regarding the implications of what to do now that their files had been encrypted.

Upon further inspection, 's.wnry', the file is another zip file. The zip file is also protected by a password however it is the same password as before.

Name	Date modified	Type
 Data	12/31/1999 3:00 PM	File folder
 Tor	12/31/1999 3:00 PM	File folder

Fig – Unzipped contents of 's.wnry'.

The two folders are used to setup Tor. The 'Data' folder is completely empty while the 'Tor' folder contains all the necessary files to run Tor. It can be assumed that 'c.wnry' (onion addresses file) and 's.wnry' are used in conjunction with each other.

Finally, 't.wnry' and 'u.wnry' are both used for the encryption and decryption of the users files respectively. 't.wnry' is encrypted making reverse engineering the encryption process impossible.

2.2.6 X86 assembly analysis

Understanding how malware performs its functions is crucial to knowing how it operates. Analysing the x86 assembly can lead to critical information being leaked about the malware. Using x32dbg, it is possible to analyse how the executable uses registers, memory addresses, instructions and more. Due to the size and scope of the executable, it is not possible to comprehensively analyse even a small fraction of the it within the debugger, however with research some noteworthy observations can be made.

00401E4F	8D85 D8FEFFFF	lea eax, dword ptr ss:[ebp-128]	
00401E55	68 10E04000	push wannacry.40E010	40E010: "c.wnry"
00401E5A	50	push eax	
00401E5B	E8 E0580000	call <JMP.&strcmp>	
00401E60	83C4 14	add esp, 14	
00401E63	85C0	test eax, eax	
00401E65	75 12	jne wannacry.401E79	
00401E67	8D85 D8FEFFFF	lea eax, dword ptr ss:[ebp-128]	
00401E6D	50	push eax	
00401E6E	FF15 68804000	call dword ptr ds:[&GetFileAttributesA]	
00401E74	83F8 FF	cmp eax, FFFFFFFF	
00401E77	75 11	jne wannacry.401E8A	
00401E79	8D85 D8FEFFFF	lea eax, dword ptr ss:[ebp-128]	
00401E7F	50	push eax	
00401E80	57	push edi	edi: "LdrpInitializeProcess"
00401E81	56	push esi	
00401E82	E8 B6570000	call wannacry.40763D	
00401E87	83C4 0C	add esp, C	
00401E8A	47	inc edi	edi: "LdrpInitializeProcess"
00401E88	3BF8	cmp edi, ebx	edi: "LdrpInitializeProcess"
00401E8D	7C B2	j1 wannacry.401E41	
00401E8F	56	push esi	
00401E90	E8 C1570000	call wannacry.407656	
00401E95	59	pop ecx	
00401E96	6A 01	push 1	
00401E98	58	pop eax	
00401E99	5B	pop ebx	
00401E9A	5F	pop edi	edi: "LdrpInitializeProcess"
00401E9B	5E	pop esi	
00401E9C	C9	leave	
00401E9D	C3	ret	
00401E9E	55	push ebp	
00401E9F	88EC	mov ebp, esp	
00401EA1	81EC 18030000	sub esp, 318	
00401EA7	8D85 E8FCFFFF	lea eax, dword ptr ss:[ebp-318]	[ebp-318]: "OLE32.DLL"
00401EAD	6A 01	push 1	
00401EAF	50	push eax	
00401EB0	C745 F4 88F44000	mov dword ptr ss:[ebp-C], wannacry.40F488	40F488: "13AM4Vw2dhxYgxeQepoHkHSQuy6NgaEb94"
00401EB7	C745 F8 64F44000	mov dword ptr ss:[ebp-8], wannacry.40F464	40F464: "12t9YDPgwueZ9NyMgw519p7AA8isjr6SMw"
00401EBE	C745 FC 40F44000	mov dword ptr ss:[ebp-4], wannacry.40F440	40F440: "11Sp7UMMngoJ1pMvKpHijcRdfjNXj6LrLn"
00401EC5	E8 36F1FFFF	call wannacry.401000	

Fig 2-15 Disassembly code focusing on 'c.wnry' and the bitcoin wallet addresses.

Firstly, it can be identified that the subprocess including 'c.wnry' is put onto the stack. This is achieved using the 'push' instruction which puts that subprocess to the top of the stack. Due to the stack operating on a Last-In-First-Out basis, the subprocess can easily be accessed and used.

A couple lines below, the function 'JMP.&strcmp' is called, this is a C function which compares 2 strings to see if they are equal. If they are, the function returns a value of 0. The WannaCry Malware Profile from Mandiant suggest that the malware checks if 'c.wnry' is the expected 0x30C [14], which could be what is occurring on these lines of code. Surprisingly, the function 'GetFileAttributes' found a few lines below do not return a value based on the files size however does provide information a malware would find useful such as whether a file is encrypted, if a file is hidden or a file which the operating system uses [17].

When running the malware (which will be elaborated upon more in dynamic analysis) one of the bitcoin wallet addresses can be seen appearing in the onion addresses file, 'c.wnry'. This behavior can be

tracked in the debugger. Using the graph feature of dbg, it becomes easier to track what the code is performing:

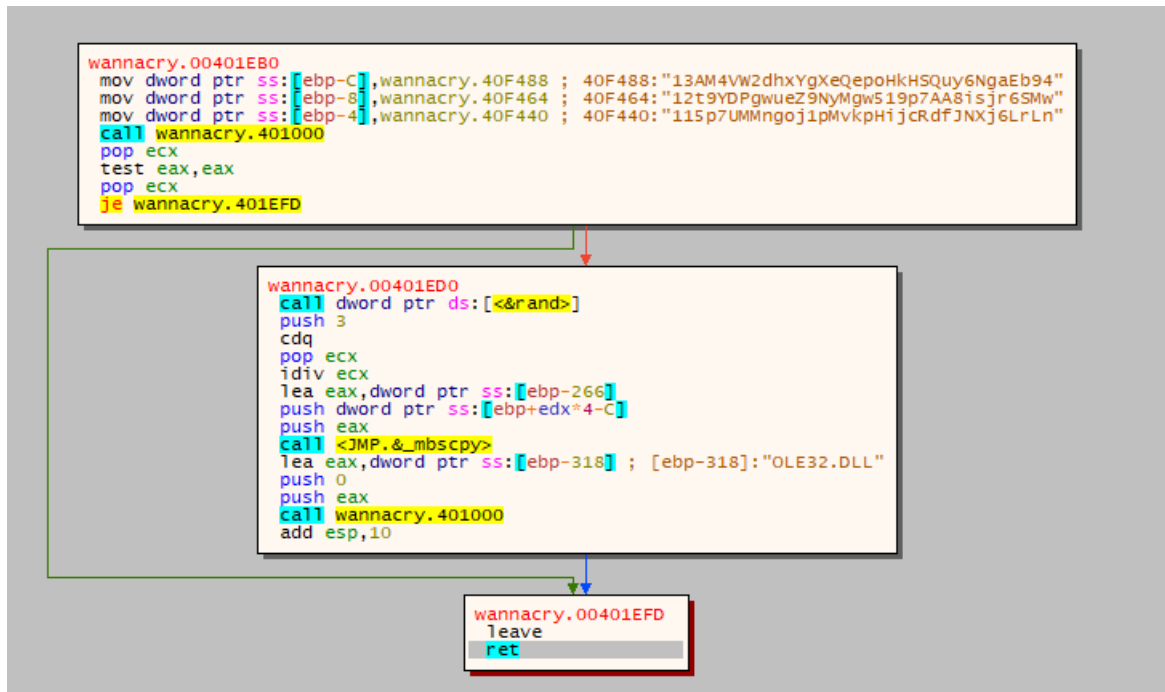


Fig 2–16 Graph featuring the bitcoin wallet addresses.

The subprocess 'wannacry.00401EB0' involves the moving of the bitcoin wallet addresses into the stack. 'dword ptr' refers to the size of what's being moved, 'ss' refers to the stack segment of memory and 'ebp-C' relates to the memory address where the subprocess will be moved to, which in this case is at the position of the stack pointer, ebp but moved back 12 bytes (C in hexadecimal is 12). Once the bitcoin addresses are loaded onto memory, the sub process 'wannacry.401000' is called.

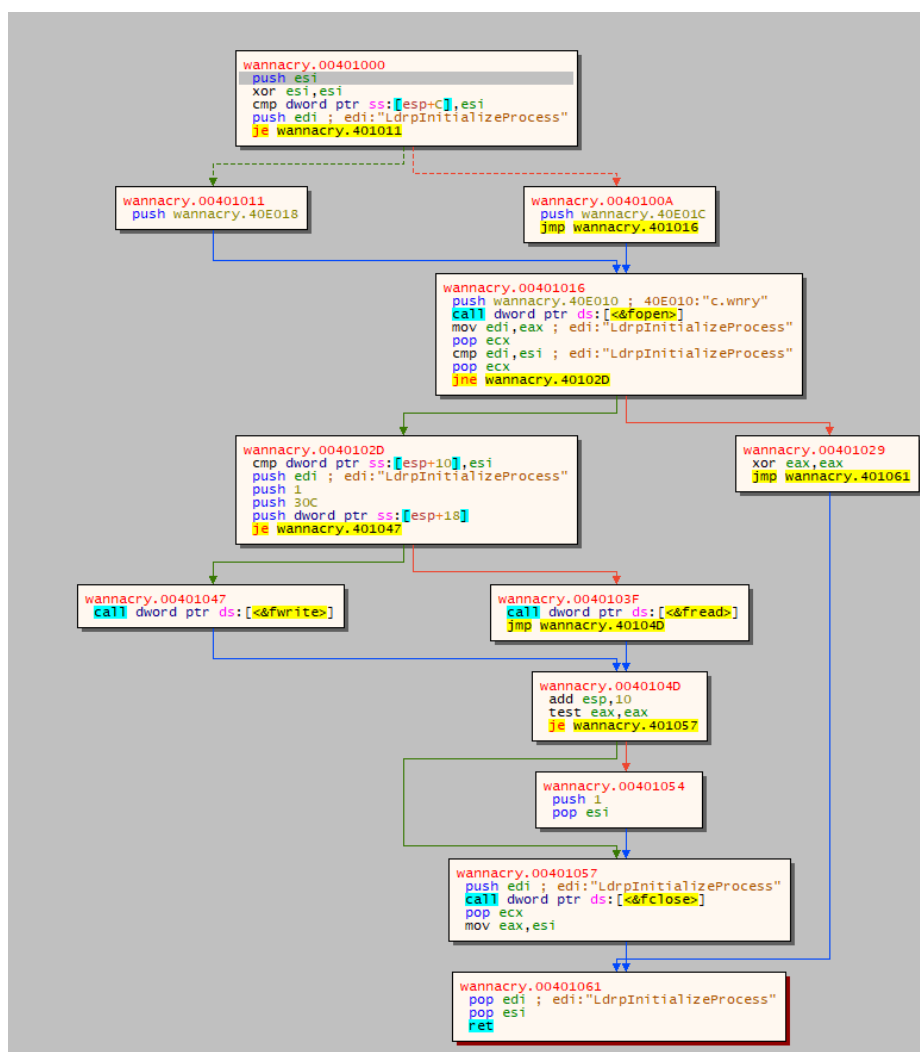


Fig 2–17 Graph featuring the file manipulation of ‘c.wnry’.

Within the subprocess, the file responsible for holding the onion addresses, ‘c.wnry’ is called. Additionally, many file manipulation functions are also called. These include:

Function name	Function purpose
Fopen	Opening of files.
Fwrite	Writing contents into files.
Fread	Reading contents of files.
fclose	Closing files.

Table 2-2 – Table including the file related functions in the graph.

From this it can be assumed that this is where a bitcoin wallet address is added to the onion configuration file.

2.3 DYNAMIC ANALYSIS

2.3.1 Running WannaCry

The WannaCry sample provided can be activated by double clicking the executable. Upon running the malware, it immediately installs numerous files to the executable's current location:

Name	Date modified	Type	Size
msg	4/24/2023 3:45 PM	File folder	
TaskData	4/24/2023 3:46 PM	File folder	
@Please_Read_Me@.txt	4/24/2023 3:45 PM	Text Document	1 KB
@WanaDecryptor@.exe	5/12/2017 2:22 AM	Application	240 KB
00000000.eky	4/24/2023 3:45 PM	EKY File	0 KB
00000000.pky	4/24/2023 3:45 PM	PKY File	1 KB
00000000.res	4/24/2023 3:46 PM	RES File	1 KB
217731682347521.bat.WNCRY	4/24/2023 3:45 PM	WNCRY File	1 KB
b.wnry	5/11/2017 8:13 PM	WNRy File	1,407 KB
c.wnry	4/24/2023 3:46 PM	WNRy File	1 KB
f.wnry	4/24/2023 3:45 PM	WNRy File	1 KB
m.vbs	4/24/2023 3:45 PM	VBScript Script File	1 KB
m.vbs.WNCRY	4/24/2023 3:45 PM	WNCRY File	1 KB
r.wnry	5/11/2017 3:59 PM	WNRy File	1 KB
s.wnry	5/9/2017 4:58 PM	WNRy File	2,968 KB
t.wnry	5/12/2017 2:22 AM	WNRy File	65 KB
taskdl.exe	5/12/2017 2:22 AM	Application	20 KB
taskse.exe	5/12/2017 2:22 AM	Application	20 KB
u.wnry	5/12/2017 2:22 AM	WNRy File	240 KB
wannacry.exe	5/14/2017 3:29 PM	Application	3,432 KB

Fig 2–18 Folder where the malware installs itself after running.

Once the malware has been ran, multiple files which were found during the manual extraction of the malware can be found again, these being most of the '.wnry' extension files. The exception to this however is 'f.wnry':

```
C:\Python39\tcl\tcl8.6\msgs\fa.msg.WNCRY
C:\Python39\tcl\tcl8.6\msgs\lv.msg.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\mingw64\lib\tcl8.6\msgs\et.msg.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\mingw64\lib\tcl8.6\msgs\th.msg.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\mingw64\lib\tk8.6\msgs\it.msg.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\usr\share\gnupg\help.tr.txt.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\usr\share\vim\vim82\doc\helphelp.txt.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\usr\share\vim\vim82\doc\spell.txt.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\usr\share\vim\vim82\doc\usr_44.txt.WNCRY
C:\Tools\Cmder\vendor\git-for-windows\usr\share\vim\vim82\doc\visual.txt.WNCRY
```

Fig 2–19 Contents of 'f.wnry' viewed in 010 editor.

Within 'f.wnry' are a list of 10 randomly selected files from the victims machine. These files are encrypted using the public key (00000000.pky) and are only decrypted if the user clicks 'decrypt' in the 'Wana Decrypt0r 2.0' popup however this is done for free [14].

The file “@Please_Read_Me@.txt” contains the same contents as “r.wnry”, informing the victim on how they can decrypt their files. “@WanaDecryptor@.exe” Appears after the malware has finished encrypting the files:



Fig 2-20 “@WanaDecryptor@.exe” pop up informing the victim on how to recover and pay the ransom.

The information on the specifics of the ransom is included here including a timer which counts down the time left until the ransom first doubles (3 days) and then the files are permanently lost (7 days). Notably the bitcoin wallet address found within the string references is found here.

The files beginning with ‘00000000’ all contribute to the encryption process. ‘00000000.pky’ is the generated public key and ‘00000000.eky’ is the generated private key.

Below the encryption management files, there is a file with a batch file followed by the ‘.WNRY’ extension. Firstly, the file name is a random selection of 15 numbers, meaning that for every time WannaCry is executed, the .bat file name will be different.

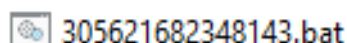


Fig 2–21 Batch file from another run of WannaCry, which has not been encrypted.

Once WannaCry has finished its encryption process and setup, the .bat file is encrypted, gaining the '.wnry' extension and creating a new file while the original is deleted. When attempting to view the batch file before it is encrypted. Viewing the contents of the batch file before its encryption was difficult however online sources can be used to show the contents:

```
@echo off
echo SET ow = WScript.CreateObject("WScript.Shell")> m.vbs
echo SET om = ow.CreateShortcut("[Full Path]\@WanaDecryptor@.exe.lnk")>> m.vbs
echo om.TargetPath = "[Full Path]\@WanaDecryptor@.exe">> m.vbs
echo om.Save>> m.vbs
cscript.exe //nologo m.vbs
del m.vbs
del /a %0
```

Fig 2–22 Contents of the batch file from WannaCry Malware Profile from Mandiant [14].

As predicted from the x86 assembly analysis, one of the bitcoin wallet addresses can be found within 'c.wnry':

```
.....-C.....  
.....  
.....  
...13AM4VW2dhxYgX  
eQepoHkHSQuy6NGa  
Eb94..  
....gx7ekbenv2ri  
ucmf.onion;57g7s  
pggrzlojinas.onio  
n;xxlvbrloxvriy2  
c5.onion;76jdd2i  
r2embyv47.onion;  
cwwnhwhlz52maqm7  
.onion;.....  
.....  
.....  
.....  
.....  
.....ht  
tps://dist.torpr  
object.org/torbro  
wser/6.5.1/tor-w  
in32-0.2.9.10.zi  
p.....
```

Fig 2-23 Bitcoin address written into 'c.wnry'.

The bitcoin address which is copied to 'c.wnry' is the one which is displayed on the WanaDecryptor popup.

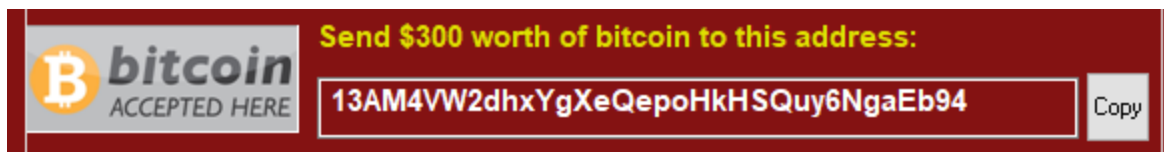


Fig 2–24 Bitcoin address written into 'c.wnry' is present on the WanaDecryptor popup.

The bitcoin address found is the one which will be used for paying the ransom. Interestingly, according to WannaCry Malware Profile from Mandiant [14], the bitcoin address should be randomly chosen from the 3 discovered in the disassembly and strings. However, upon testing the first bitcoin address is always put into 'c.wnry' along with the amount the memory address was moved from the stack pointer. '-C'.

2.3.2 Function analysis

Malware must use functions to carry out its actions and further cause damage to the victims' machine. It is possible to analyse what functions the malware is using to develop a better understanding of how it operates. When running WannaCry, it is possible to view all processes being started and exited using ProcMon (process monitor).

Wannacry.exe (5488)	DiskPart	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... "C:\Users\user\Desktop\Samples\1\1\wannacry.exe"
attrib.exe (1824)	Attribute Utility	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... attrib +h .
conhost.exe (2520)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
icacls.exe (4956)	C:\Windows\Sys...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... icacls . /grant Everyone:F /T /C /Q
conhost.exe (3048)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
taskd.exe (5624)	SQL Client Config...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... taskd.exe
cmd.exe (4276)	Windows Comma...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... C:\Windows\system32\cmd.exe /c 243581682438013.bat
conhost.exe (2108)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
cscript.exe (3928)	Microsoft © Conso...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... cscript.exe //nologo m.vbs
@WanaDecryptor@.exe (2108)	Load PerfMon Co...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... @WanaDecryptor@.exe co
taskhsvc.exe (2832)	C:\Users\user\De...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... TaskData\Tor\taskhsvc.exe
conhost.exe (4776)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
cmd.exe (4368)	Windows Comma...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... cmd.exe /c start /b @WanaDecryptor@.exe vs
conhost.exe (2812)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
@WanaDecryptor@.exe (2812)	Load PerfMon Co...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... @WanaDecryptor@.exe vs
@WanaDecryptor@.exe (2812)	Load PerfMon Co...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... @WanaDecryptor@.exe vs
WerFault.exe (1408)	Windows Problem...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... C:\Windows\SysWOW64\WerFault.exe -u -p 4856 -s 960
@WanaDecryptor@.exe (3472)	Load PerfMon Co...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... @WanaDecryptor@.exe vs
WerFault.exe (3472)	Windows Problem...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... C:\Windows\SysWOW64\WerFault.exe -u -p 4856 -s 696
taskd.exe (644)	SQL Client Config...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... taskd.exe
@WanaDecryptor@.exe (4368)	Load PerfMon Co...	C:\Users\user\De...	Microsoft Corporat... DESKTOP-14QC1... @WanaDecryptor@.exe
cmd.exe (3888)	Windows Comma...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... cmd.exe /c reg add HKCU\SOFTWARE\Microsoft\Windows\...
conhost.exe (2464)	Console Window ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... \??\C:\Windows\system32\conhost.exe 0xffffffff -ForceV1
reg.exe (248)	Registry Console ...	C:\Windows\Sys...	Microsoft Corporat... DESKTOP-14QC1... reg add HKCU\SOFTWARE\Microsoft\Windows\CurrentVersi...

Fig 2–25 Using ProcMon function tree after running WannaCry.

When running WannaCry, numerous processes are created and exited between the time of starting the executable and the malware has finished its setup, when '@WanaDecryptor' process is created. The first process identified is 'attrib.exe', which is responsible for changing the attributes of certain files. It makes use of the 'conhost.exe' which allows for applications to handle their command line arguments [18]. The command the process is utilising is 'attrib +h' which makes the file appear as hidden [19]. This command is performed on the file path where the malware is located.

Next 'icacls.exe' runs the command '. /grant Everyone:F /T /C /Q' which is used to grant permissions to everyone [20]. The command can be broken down as follows:

Syntax name	Description
Grant Everyone	Gives the permissions to everyone.
/T	Access to all files.
/C	Refuse to be stopped by file errors/access denied pop ups.
/Q	Does not alert the user of any success messages.

Table 2-3 – Table containing the different parameters of the command.

This confirms the malware's ability to take malicious control over the user's machine.

Below is a summary of the other processes in the malware:

Process name	Description
Taskdl.exe	Performs general clean up including deleting certain files once the malware has finished initialising.
Cmd.exe (4276)	Runs the .bat file.
Cscript.exe	Runs the m.vbs file.
Cmd.exe (3888)	Adds registers.

Table 2-4 – Table containing a summary of the other functions identified.

2.3.3 Network analysis

Observing how a malware interacts with the environment outside of the hosts PC is crucial to developing an understanding of it. Malware will often want to propagate to other machines to further spread its influence and cause more users to be victim to ransomware. Wireshark was used to analyse packets being sent from the machine while the malware was being executed. Due to the test environment not being connected to any network for safety reasons, the test was performed without using a fake network:

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
2	0.000527	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
3	0.704291	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
4	0.704767	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
5	1.469637	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
6	1.470106	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
7	2.988705	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
8	2.989165	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
9	3.735602	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
10	3.736357	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
11	4.486038	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
12	4.486617	192.168.1.128	192.168.1.255	NBNS	92	Name query NB DESKTOP-14QC1L8<1c>
13	10.201401	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
14	10.270282	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
15	10.301863	169.254.28.135	239.255.255.250	SSDP	211	M-SEARCH * HTTP/1.1
16	11.215935	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
17	11.278935	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
18	11.310193	169.254.28.135	239.255.255.250	SSDP	211	M-SEARCH * HTTP/1.1
19	12.230548	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1
20	12.294424	169.254.28.135	239.255.255.250	SSDP	217	M-SEARCH * HTTP/1.1

Fig 2–26 Wireshark output while running the sample.

The 2 protocols being identified both hint towards WannaCry attempting to discover more machines. The NBNS protocol, or 'NetBIOS Name Service', attempts to look for a host on an internal network in scenarios where the host's IP address cannot be identified using the DNS server [21]. The other protocol

is SSDP (Simple Service Discovery Protocol), which is used for advertising and identifying in small networks, often home networks. Evaluating this, WannaCry is attempting to use these protocols to find new hosts within the network to spread to.

3 DISCUSSION

3.1 OVERALL DISCUSSION

The malware analysis performed on sample 1 was effective and successful with numerous characteristics, functions and code leading to the belief that the sample was a part of the well-known malware, WannaCry. This malware is incredibly dangerous and is known for causing the NHS \$100 million in damages [22]. During the attack, malware analysis was performed, similar to the one in this report, further reinforcing the necessity for malware analysis. The methodology used was very effective, going from static analysis to dynamic analysis. Starting with static analysis allowed for more research and evaluation to take place before running the malware in a test environment.

3.1.1 Static analysis

The static analysis performed on the sample made it noticeably clear that it was WannaCry. Utilising VirusTotal was highly effective in providing a straightforward way of a starting point to begin analysis. While VirusTotal was correct in assuming it was WannaCry, it was not enough to rely on a singular source to make an assumption due to false positives being a potentiality.

The DLL and strings analysis provided the first confirmations from looking purely into the executable file that the sample was indeed WannaCry. Identifying the DLLs which the malware uses allowed for the idea that this sample can manipulate memory and files while also controlling the user's interactivity. Additionally, the 'MSVCRT.DLL' was the first sign of the malware being packed. The strings analysis (especially when used in conjunction with a debugger) provided excellent information regarding the nature of the malware and its capabilities. Some of the most crucial information to this investigation was found within the string references, including many starting points. The password being used for the zip files and bitcoin wallet addresses helped to establish some of the inner mechanisms of the malware, which matched with the expected behaviour of WannaCry, further reinforcing the belief that it is present in the sample.

Using the password found within the string references and identifying that the executable file could be unzipped, the contents of the sample were extracted. Within this folder, many files relating to the setup and configuration of the malware were identified. This included encryption and decryption management files, ransom information and image, and Tor configuration files. Additionally, many of these files ended in the '.wnry' extension which is exclusive to WannaCry.

The x86 disassembly analysis was effective in predicting what the malware would accomplish during dynamic analysis. One behaviour which was able to be predicted using the debugger was the bitcoin wallets being copied to the onion address folder, which would later be used to setup Tor and the C2 communication servers. More insight was given to the other functions the malware uses such as file manipulation functions.

3.1.2 Dynamic analysis

The dynamic analysis built upon information acquired during the static analysis. Firstly, observing how the malware behaved when the executable was ran, was a crucial step in the analysis. Evaluating the various files created and processes being ran allowed for key characteristics to be identified. The files

found during the zip file extraction process and string reference searching could be found within when running the malware. Interesting, a few files which were not present during the extraction process did appear such as files which contribute to the RSA encryption process and code files which are responsible for the management of computer system files. Additionally, the behaviour which was predicted using the debugger was confirmed with the first bitcoin wallet address appearing alongside the onion addresses.

Utilising ProcMon to analyse what functions are being created and exited was extremely beneficial to the investigation. The results elaborated on the malwares ability to control a user's machine through using functions to hide files or grant permission to itself to manipulate any files. Other functions which were responsible for the clean-up and deletion of certain WannaCry related files were identified as well.

Network analysis proved to be useful in identifying how WannaCry will attempt to propagate itself through the network, even while a connection to a DNS server is impossible. WannaCry achieves this by utilising 2 types of protocols, NBNS and SSDP, and uses them to find more victims' machines to hold ransom.

3.2 COUNTERMEASURES

Despite the threat of WannaCry appearing impossibly dangerous, many countermeasures can be taken to protect machines from it and other types of malware. Firstly, using a suitable anti-virus software will greatly protect a machine from malware, however researching the best ones is critical. For example, when uploading the hash of the executable file sample of WannaCry, not all security vendor anti-virus software identified it as being malicious, let alone specifically WannaCry. Additionally, making sure the machine's version of windows is up to date is also critical due to WannaCry relying on a pre-existing exploit which is patched in recent updates.

Despite this version of WannaCry not carrying the worm part of it, understanding how to best identify and counteract worm activity on a machine is crucial. Tell-tale signs of worm activity on a machine include [23]:

- Missing or hidden files/folders.
- Slow performance.
- Irregular emails being sent.

If believed to currently be infected with a worm necessary actions should be taken such as disconnecting the machine from the network and removing the worm.

The most common method for WannaCry to begin propagating throughout a network is through a trojan. A trojan disguises itself as a harmless and helpful file when in reality it is carrying malware. The most effective way to combat trojans is to only open files from a trusted source and never download content from unofficial sources.

3.3 FUTURE WORK

Due to time constraints, only a limited amount of malware analysis could be performed on the sample. Overall, many smaller aspects of the procedure could have been more in depth. A greater look into what the functions of the DLL libraries could have been very beneficial as these functions could have further hinted at how WannaCry exactly performs its exploits. The x86 disassembly analysis was sufficient but the time taken to identify an expected behaviour of the malware took an unreasonable amount of time to accomplish. Adding to this, prior knowledge of how disassembly code works was not acceptable and should be improved. Additionally, investigating how the malware utilises registers would have been helpful.

More in-depth function analysis should have been performed including going into detail of file locations where the functions take place and how exactly they can cause damage to a machine. Greater knowledge on how windows machines behave and complete tasks on a low level would have greatly contributed to furthering the function analysis.

One aspect which could have greatly been more thoroughly analysed was the networking analysis. Due to the worm portion of the malware not being present in the sample and the test environment blocking any outside network communications as a safety precaution, not much extensive network testing could be performed. A work around this is using tools which simulate a fake network. This was attempted however the results were not conclusive and was hard to determine what the malware was trying to accomplish resulting in that analysis being left out.

4 CONCLUSION

The malware analysis was a success with the chosen sample being identified as the malicious malware known as WannaCry. The reasoning behind this identification was the static and dynamic analysis provided sufficient evidence which also coincided alongside online resources. This WannaCry analysis report can be beneficial in identifying other strands of malware by utilising similar techniques and/or methodologies. Additionally, the countermeasures provided should be universally used regardless of the type of malware.

5 BIBLIOGRAPHY

- [1] Griffiths, C. (2023, April). *The latest ransomware statistics*. Retrieved April 2023, from aag: <https://aag-it.com/the-latest-ransomware-statistics/#:~:text=How%20many%20malware%20attacks%20were,%2C%20viruses%2C%20trojans%20and%20worms.>
- [2] Heaslip, E. (2023, January 23). *What Small Businesses Need To Know About Ransomware*. Retrieved April 2023, from uschamber: <https://www.uschamber.com/co/run/technology/small-businesses-ransomware>
- [3] Jovanovic, B. (2023, April 7). *A Not-so-Common Cold: Malware Statistics in 2023*. Retrieved April 2023, from Dataprot: <https://dataprot.net/statistics/malware-statistics/#:~:text=There%20are%20more%20than%201%20billion%20malware%20programs%20out%20there.&text=Even%20with%20built%20in%20antivirus,malware%20online%20than%20ever%20before.>
- [4] AV test. (2023, April). *Malware*. Retrieved April 2023, from AV test: <https://www.av-test.org/en/statistics/malware/>
- [5] vmware. (n.d.). *What is Malware Analysis?* Retrieved April 2023, from vmware: <https://www.vmware.com/topics/glossary/content/malware-analysis.html#:~:text=Malware%20Analysis%20is%20important%20because,causing%20destruction%20within%20the%20organization.>
- [6] Baker, K. (2023, April 17). *Malware analysis*. Retrieved April 2023, from CrowdStrike: <https://www.crowdstrike.com/cybersecurity-101/malware/malware-analysis/>
- [7] Nationwide. (n.d.). *Understanding, recognizing and preventing malware*. Retrieved April 2023, from Nationwide: <https://www.nationwide.com/lc/resources/personal-finance/articles/how-malware-works#:~:text=The%20term%20%22malware%22%20refers%20to,an%20electronic%20device's%20normal%20operation.&text=Malware%20can%20infect%20personal%20computers,developed%20was%20the%20>
- [8] VirusTotal. (n.d.). *How it works*. Retrieved April 2023, from VirusTotal: <https://support.virustotal.com/hc/en-us/articles/115002126889-How-it-works#:~:text=VirusTotal%20inspects%20items%20with%20over,and%20send%20it%20to%20VirusTotal.>
- [9] Kaspersky. (n.d.). *What is WannaCry ransomware?* Retrieved April 2023, from Kaspersky: <https://www.kaspersky.com/resource-center/threats/ransomware-wannacry>
- [10] Microsoft. (2023, April 29). *What is a DLL*. Retrieved April 2023, from Microsoft: <https://learn.microsoft.com/en-us/troubleshoot/windows-client/deployment/dynamic-link-library>

- [11] Rouse, M. (2016, February 1). *Kernel32.dll*. Retrieved April 2023, from Technopedia: <https://www.techopedia.com/definition/3379/kernel32dll>
- [12] Process Library. (n.d.). *user32.dll*. Retrieved April 2023, from Process Library: <https://www.processlibrary.com/en/directory/files/user32/19597/>
- [13] Process Library. (n.d.). *msvcrt.dll*. Retrieved April 2023, from Process Library: <https://www.processlibrary.com/en/directory/files/msvcrt/20015/>
- [14] Berry, A., Homan, J., & Eitzman, R. (2021, November 5). *WannaCry Malware Profile*. Retrieved March 2023, from Mandiant: <https://www.mandiant.com/resources/blog/wannacry-malware-profile>
- [15] Murtaza, F. (2023, January 27). *How tor addresses work*. Retrieved April 2023, from Make use of: <https://www.mandiant.com/resources/blog/wannacry-malware-profile>
- [16] Frankenfield, J. (2022, September 23). *What Is Tor? Who Uses It, How to Use It, Legality, and Purpose*. Retrieved April 2023, from Investopedia: <https://www.investopedia.com/terms/t/tor.asp>
- [17] Microsoft. (2023, February 9). *GetFileAttributesA function (fileapi.h)*. Retrieved April 2023, from Microsoft: <https://learn.microsoft.com/en-us/windows/win32/api/fileapi/nf-fileapi-getfileattributesa>
- [18] Glenn, W. (2022, December 22). *What Is conhost.exe and Why Is It Running?* Retrieved April 2023, from How-To Geek: <https://www.howtogeek.com/4996/what-is-conhost.exe-and-why-is-it-running>
- [19] Aurelie. (2023, March 28). *What Is Attrib Command & How to Use It in Windows 10/11? [MiniTool Tips]*. Retrieved April 2023, from MiniTool: <https://www.minitool.com/backup-tips/attrib-command.html>
- [20] SS64. (n.d.). *ICACLS.exe (2003 sp2, Vista+)*. Retrieved April 2023, from ss64: <https://ss64.com/nt/icacls.html>
- [21] Miller, M. (n.d.). *Vulnerability Walkthrough – NBNS And LLMNR Spoofing*. Retrieved April 2023, from Triaxiom Security: <https://www.triaxiomsecurity.com/vulnerability-walkthrough-nbns-and-llmnr-spoofing/>
- [22] Brunua, C. (2018, October 18). *Ransomware News: WannaCry Attack Costs NHS Over \$100 Million*. Retrieved April 2023, from datto: <https://www.datto.com/uk/blog/ransomware-news-wannacry-attack-costs-nhs-over-100-million#:~:text=Ransomware%20News%3A%20WannaCry%20Attack%20Costs%20NHS%20Over%20%24100%20Million,-By%20Chris%20Brunau&text=According%20to%20estimates%20from%20the,and%20resto>
- [23] Vigderman, A., & Turner, G. (27, February 2023). *What Is a Computer Worm?* Retrieved April 2023, from security: <https://www.security.org/antivirus/computer-worm/>

APPENDICES PART 1

APPENDIX A

```
<assembly xmlns="urn:schemas-microsoft-com:asm.v1" manifestVersion="1.0">
  <trustInfo xmlns="urn:schemas-microsoft-com:asm.v2">
    <security>
      <requestedPrivileges>
        <requestedExecutionLevel level="asInvoker" />
      </requestedPrivileges>
    </security>
  </trustInfo>
  <dependency>
    <dependentAssembly>
      <assemblyIdentity
        type="win32"
        name="Microsoft.Windows.Common-Controls"
        version="6.0.0.0"
        processorArchitecture="*"
        publicKeyToken="6595b64144ccf1df"
        language="*"
      />
    </dependentAssembly>
  </dependency>
  <compatibility xmlns="urn:schemas-microsoft-com:compatibility.v1">
    <application>
      <!-- Windows 10 -->
      <supportedOS Id="{8e0f7a12-bfb3-4fe8-b9a5-48fd50a15a9a}"/>
      <!-- Windows 8.1 -->
      <supportedOS Id="{1f676c76-80e1-4239-95bb-83d0f6d0da78}"/>
      <!-- Windows Vista -->
      <supportedOS Id="{e2011457-1546-43c5-a5fe-008deee3d3f0}"/>
      <!-- Windows 7 -->
      <supportedOS Id="{35138b9a-5d96-4fbd-8e2d-a2440225f93a}"/>
      <!-- Windows 8 -->
      <supportedOS Id="{4a2f28e3-53b9-4441-ba9c-d69d4a4a6e38}"/>
    </application>
  </compatibility>
</assembly>
```

PE studio manifest